

Guía de Desarrollo: Marca Personal FP (Parte 3)

Continuación de los Bloques 1-4. Misma estructura en cada apartado:

- ¿Qué es? → Teoría mínima.
- Ejemplo genérico → Código de muestra (tienda de **Videojuegos**). **NO lo copies al proyecto.**
- Reto del proyecto → Lo que TÚ programas. Te doy comandos y guía, **no el código.**

Requisito previo: Controladores y rutas del Bloque 4 funcionando.

BLOQUE 5: Vistas Blade

5.1. Layouts y directivas de Blade

¿Qué es y para qué sirve?

- **Blade** es el motor de plantillas de Laravel. Mezcla **HTML + directivas** (`@...`) y evita repetir código.
- Un **layout** es la "plantilla madre" (cabecera, menú, pie). Las vistas hijas **rellenan los huecos**.
- Directivas clave:
 - **@extends** → la vista hija dice de qué layout hereda.
 - **@yield** → en el layout, marca un **hueco** a rellenar.
 - **@section** → en la hija, **rellena** ese hueco.
 - **@foreach** → recorre una colección (listados).

Atención (Breeze): El layout que instaló Breeze usa **componentes** (`<x-app-layout>`), no `@extends/@yield`. Para el proyecto puedes **crear tu propio layout clásico** con `@extends/@yield` como pide el currículum, o reutilizar el de Breeze. No mezcles los dos estilos en la misma vista.

Ejemplo genérico (tienda de Videojuegos)

Layout madre `resources/views/layouts/app.blade.php`:

None

```
<!DOCTYPE html>
<html lang="es">
<head>
    <title>@yield('titulo', 'Tienda')</title>
</head>
<body>
    <nav>Mi menú</nav>
    <main>
        @yield('contenido')
    </main>
</body>
</html>
```

Vista hija `resources/views/videojuegos/index.blade.php`:

None

```
@extends('layouts.app')

@section('titulo', 'Catálogo')

@section('contenido')
    <h1>Videojuegos</h1>

    @forelse ($videojuegos as $juego)
        <p>{{ $juego->titulo }} - {{ $juego->precio }} €</p>
    @empty
        <p>No hay videojuegos.</p>
    @endforelse

    {{ $videojuegos->links() }}
@endsection
```

Pistas: `{{ $variable }}` imprime de forma segura (escapa HTML). `@forelse/@empty` es un `@foreach` con caso "lista vacía". `->links()` pinta la paginación.

Reto del proyecto

1. Crea tu layout en `resources/views/layouts/app.blade.php` con, al menos:
 - Un `@yield('titulo')` en el `<title>`.

- Un `@yield('contenido')` para el cuerpo.
 - Un menú con enlaces usando el helper `route()` (ej. `route('proyectos.index')`).
2. **Crea las vistas de listado** (`index`) que extienden el layout:

Vista	Carpeta sugerida	Qué recorre con <code>@foreach</code>
Listado de proyectos	<code>proyectos/index.blade.php</code>	La colección <code>\$proyectos</code>
Listado de estudiantes	<code>estudiantes/index.blade.php</code>	La colección <code>\$estudiantes</code>
Listado de ciclos	<code>ciclos/index.blade.php</code>	La colección <code>\$ciclos</code>

3. **Crea la vista de detalle** (`show`) para mostrar un registro y **sus relaciones** (ej. los estudiantes de un proyecto con otro `@foreach`).

Pistas importantes:

- Muestra datos relacionados recorriendo la relación: `@foreach ($proyecto->estudiantes as $est) ... @endforeach`.
- Usa `@if` / `@empty` para los casos sin datos.

Objetivo: Navegar por los listados y el detalle, viendo datos reales y relaciones.

5.2. Formularios (`@csrf`, `@method`)

¿Qué es y para qué sirve?

- Un **formulario** envía datos al servidor (crear / editar). Apunta a una **ruta** y usa un **método HTTP**.
- `@csrf` → token de seguridad **obligatorio** en todo formulario `POST`. Sin él, Laravel rechaza la petición (error 419).
- `@method('PUT')` / `@method('DELETE')` → el HTML solo sabe `GET` y `POST`. Esta directiva **simula** PUT/PATCH/DELETE para las rutas resource (`update`, `destroy`).

Ejemplo genérico (tienda de Videojuegos)

Formulario de **creación** (`store`):

None

```
<form action="{{ route('videojuegos.store') }}" method="POST">
    @csrf

    <input type="text" name="titulo" value="{{ old('titulo') }}">
    @error('titulo') <span>{{ $message }}</span> @enderror

    <input type="number" name="precio" value="{{ old('precio')
    }}">

    <button type="submit">Guardar</button>
</form>
```

Formulario de **edición** (update), fíjate en @method(' PUT '):

None

```
<form action="{{ route('videojuegos.update', $videojuego) }}"
method="POST">
    @csrf
    @method( ' PUT ' )

    <input type="text" name="titulo" value="{{ old('titulo',
    $videojuego->titulo) }}">

    <button type="submit">Actualizar</button>
</form>
```

Botón de **borrado** (destroy):

None

```
<form action="{{ route('videojuegos.destroy', $videojuego) }}"
method="POST">
    @csrf
    @method( ' DELETE ' )
    <button type="submit">Eliminar</button>
</form>
```

Pistas: `old( ' campo ' )` rellena el formulario con lo que el usuario escribió si la validación falla. `@error( ' campo ' )` muestra el mensaje de error de ese campo.

Reto del proyecto

1. **Crea los formularios** de crear y editar para tus entidades:

Vista	Método del form	Directiva extra
proyectos/create.blade.php	POST → ruta store	solo @csrf
proyectos/edit.blade.php	POST → ruta update	@csrf + @method(' PUT ')
estudiantes/create.blade.php	POST → ruta store	solo @csrf
estudiantes/edit.blade.php	POST → ruta update	@csrf + @method(' PUT ')

- 2.

Añade el botón de borrado (`destroy`) con `@csrf + @method( ' DELETE ' )` en los listados o el detalle.

3. **Gestiona los campos de relación (N:M):**

- Para asignar estudiantes/docentes a un proyecto, usa un `<select multiple name="estudiantes[]">` recorrido con `@foreach`.
- En el controlador/servicio, vincula con `sync()`.

4. **Muestra errores y conserva datos:** usa `@error` y `old()` en cada campo.

Pistas importantes:

- El método del `<form>` siempre es `POST`; lo que cambia es `@method( ... )`.
- Recuerda **validar** en el controlador antes de guardar (Bloque 4).

Objetivo: Crear, editar y borrar registros desde el navegador, con validación y relaciones funcionando.

Has terminado el Bloque 5 y el recorrido base del proyecto. Tienes la aplicación completa de extremo a extremo: configuración, autenticación, base de datos, dominio, controladores y vistas con CRUD.